LLM 里的扩散模型（Diffusion Model）是什么？

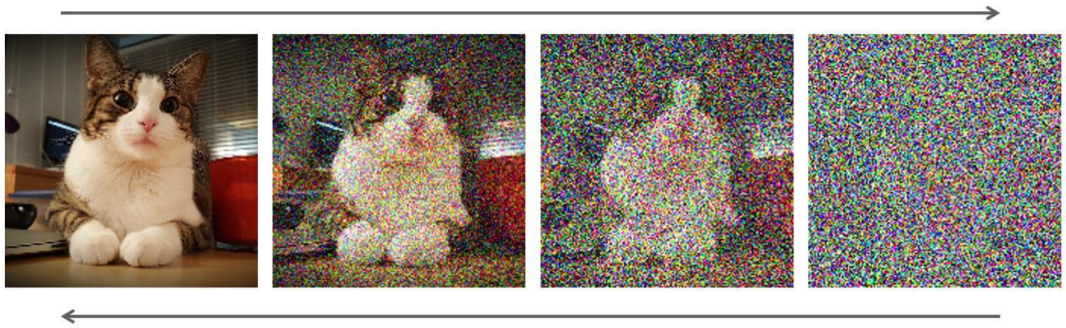
扩散模型是一类通过“逐步加噪 → 学习去噪 → 逐步生成”来产生数据的生成模型。

它最早在图像生成领域大放异彩，例如 Stable Diffusion、DALL·E。现在也被用于文本生成、语音、视频和其他模态。

你最近在学习 Transformer、Qwen 和 LLM，可以把扩散模型理解成：和传统自回归 LLM 不同的一种生成方式。

1. 先理解扩散模型的核心原理

想象一张清晰的猫咪图片：



扩散模型训练时做两件事：

正向扩散：不断加噪声

从一张真实图片开始，每一步添加一点随机噪声。

清晰图片
↓ 加一点噪声
有点模糊
↓ 继续加噪声
很模糊
↓ 继续加噪声

几乎全是噪声
↓
纯随机噪声

数学上可以写成：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

其中：

x0 ：原始清晰数据。

xt ：第 t 步的带噪数据。

ϵ：随机高斯噪声。

α¯t ：控制原始数据还剩多少。

当 t 足够大时，图片就接近纯随机噪声。

反向扩散：逐步去噪生成

训练一个神经网络，学习如何从带噪数据中恢复有意义的结构。

纯随机噪声
↓ 去噪一步
模糊轮廓
↓ 去噪一步
猫的轮廓
↓ 去噪一步
猫的身体和脸
↓ 去噪一步
清晰猫咪图片

生成时从随机噪声开始，经过多次去噪，得到最终结果。

关键：模型不是直接从零“画出”图片，而是学习如何把噪声变成有结构的数据。

2. 那么 LLM 里的扩散模型是什么意思？

传统 LLM，比如 GPT、很多 Qwen 模型，主要使用自回归生成（Autoregressive Generation）。

扩散式语言模型则尝试用类似扩散的方式生成文本。

传统 LLM：一个 Token 接一个 Token

假设你输入：

今天天气

模型依次预测：

```
今天天气
    ↓
今天天气很好
    ↓
今天天气很好，适合
    ↓
今天天气很好，适合出去
    ↓
今天天气很好，适合出去散步
```

每次预测下一个 Token。

数学形式：

$$P(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} P(x_i | x_{<i})$$

也就是根据前面的 Token 预测下一个 Token。

扩散式 LLM：逐步恢复整段文本

假设目标文本是：

今天天气很好，适合出去散步。

训练时可能将部分 Token 替换成 `[MASK]` 或噪声表示：

```
[MASK] 天气 [MASK]，适合 [MASK] 散步。
```

模型通过学习恢复被破坏的 Token：

```
今天天气很好，适合出去散步。
```

在生成时，可以从大量被掩码的文本表示开始，逐步恢复内容。

```
[MASK] [MASK] [MASK] [MASK] [MASK]
                ↓
[MASK] 天气 [MASK] [MASK] 散步
                ↓
今天天气很好，适合出去散步
```

这就是扩散式语言模型的直观理解。

不过要注意：文本不是连续图片像素，LLM 里的扩散模型通常需要专门的离散扩散或连续表示方法。

3. 传统 LLM 和扩散 LLM 的区别

对比	自回归 LLM	扩散式 LLM
代表思路	GPT、Qwen 的传统生成方式	Diffusion Language Model
生成方式	从左到右逐 Token 生成	逐步去噪/恢复 Token
训练目标	预测下一个 Token	恢复被破坏的 Token 或噪声
是否依赖严格左到右	通常是	不一定
生成过程	Token 1 → Token 2 → Token 3	多轮迭代修复整段
擅长特点	流式生成、成熟的文本生成	可能更适合并行生成、全局修改
主要问题	长文本生成存在串行依赖	多轮去噪成本、离散文本建模难度

一个非常重要的区别：生成速度

传统 LLM：

```
Token 1 → Token 2 → Token 3 → Token 4 → Token 5
```

扩散式 LLM：

```
整段文本的初始状态
        ↓
   多个 Token 一起修复
        ↓
   多个 Token 一起修复
        ↓
   最终文本
```

扩散模型理论上可以减少严格的逐 Token 依赖，存在并行生成的潜力。

但不是说扩散模型一定比 GPT 快。实际速度还取决于去噪步数、模型规模、采样算法和硬件。

4. LLM 里的扩散模型到底有哪些类型？

这里是理解论文时比较重要的部分。

A. 离散扩散语言模型（Discrete Diffusion LM）

文本由离散 Token 构成，例如：

```
我 爱 学习 AI
```

可以将 Token 替换成 `[MASK]` 或其他离散状态。

训练模型学习：

$$p_\theta(x_{t-1}|x_t)$$

意思是：根据当前状态 xtx_txt ，预测更接近原始文本的状态 xt−1x_{t-1}xt−1 。

代表研究方向包括：

Masked Diffusion Language Model

Discrete Diffusion Language Model

Masked Generative Transformer

B. 连续扩散语言模型（Continuous Diffusion LM）

不直接在离散 Token 上扩散，而是先把 Token 转换成连续向量。

```
Token
  ↓
Embedding 向量
  ↓
连续扩散过程
  ↓
去噪后的向量
  ↓
映射回 Token
```

例如 Token embedding：

$$e_i \in \mathbb{R}^d$$

在连续空间中进行加噪和去噪。

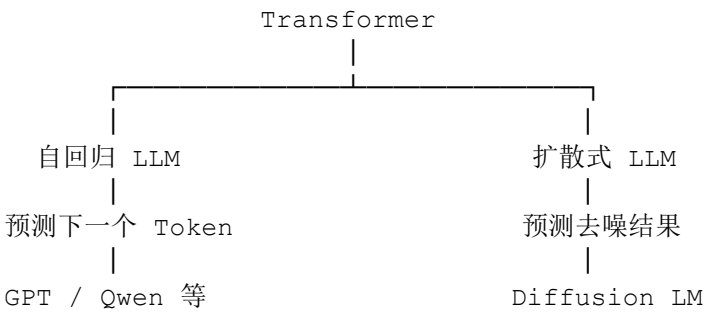
这和图像扩散模型更接近，但最终仍然需要得到离散文本。

5. 它和 Transformer 是什么关系？

扩散模型不是 Transformer 的替代品，而是生成方式/训练框架上的不同。

Transformer 可以用来做扩散模型的去噪网络。

例如：



所以扩散模型也可能使用：

Multi-Head Attention

Self-Attention

Feed Forward Network

RoPE

Transformer Block

区别主要在于：Transformer 每次输出的训练目标和生成流程不同。

6. 和 Qwen 有什么关系？

你之前问过 Qwen 的结构，这里可以这样理解：

传统 Qwen LLM

```
输入 Token
    ↓
Embedding
    ↓
Transformer Blocks
    ↓
LM Head
    ↓
预测下一个 Token
```

扩散式语言模型

```
带噪文本 / Masked Token
    ↓
Embedding
```

```
    ↓
Transformer Blocks
    ↓
去噪预测头
    ↓
恢复 Token 或连续表示
    ↓
重复多轮
    ↓
生成完整文本
```

因此：

Qwen 传统架构的核心是自回归语言建模。

扩散式 LLM 可以继续使用 Transformer。

两者的主要差异不在于“有没有 Attention”，而在于训练目标、噪声过程和生成算法。

7. 为什么要研究扩散式 LLM？

传统自回归模型有一个天然限制：

生成很长的文本时，Token 之间存在严格的串行依赖。

比如生成 1000 个 Token，传统方法通常需要依次生成这些 Token。

扩散式 LLM 希望：

并行生成多个 Token：减少串行依赖。

更容易全局修改：不必只从左到右。

更适合某些规划任务：先形成整体，再逐步完善。

探索新的生成质量与速度权衡。

但目前不能简单认为它全面优于 GPT/Qwen。传统自回归 LLM 的训练、推理、工具调用和生态都非常成熟。

8. 一个程序员容易理解的类比

假设你要生成一段 Python 代码：

```
def fibonacci(n):
    ...
```

自回归 LLM

像程序员逐字写代码：

```
defdef fibonaccidef fibonacci(n)def fibonacci(n):def fibonacci(n):
    if
...
```

每次根据前面的代码继续写。

扩散式 LLM

像程序员先写出一份不完整的代码草稿：

```
def fibonacci(n):
    [MASK]
    [MASK]
    return ...
```

然后多次检查和修复：

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

这种方式更像先形成整体草稿，再逐步修正。

最后总结

你可以记住这句话：

> 传统 LLM 是“从左到右预测下一个 Token”；扩散式 LLM 是“从噪声或不完整文本出发，通过多轮去噪/恢复生成完整文本”。

它们都可以使用 Transformer，区别主要是生成机制。

Diffusion Language Model 深度教程

这次我们从数学原理、训练目标、Transformer 架构和代码实现四个层面，系统理解 Diffusion Language Model（扩散语言模型，简称 Diffusion LM）。

你可以把它当作一篇从传统 Qwen 自回归模型过渡到扩散式 LLM 的入门技术教程。重点是理解：为什么文本生成可以不再严格地从左到右，以及模型究竟在训练什么。

一、先建立整体认知

1. 两种 LLM 的核心区别

A. Qwen：自回归语言模型

Autoregressive Language Model

输入：我喜欢

↓ 预测下一个 Token

吃

↓ 继续预测

苹果

↓ 继续预测

。

每次生成下一个 Token，已经生成的内容通常作为后续上下文。

B. Diffusion LM：扩散式语言模型

以 Masked Diffusion 为例

[MASK] [MASK] [MASK] [MASK]

↓ 并行预测和更新

我 [MASK] 喜欢 [MASK]

↓ 继续迭代修复

我 喜欢 吃 苹果

先有一段待恢复的文本表示，再通过多轮更新生成完整序列。

这里先限定一个重要范围：下面主要讲 Masked Discrete Diffusion LM（掩码式离散扩散语言模型）。它是理解扩散式 LLM 最容易入门的一类。连续扩散语言模型和其他离散扩散方法会在后面区分。

二、为什么文本也可以做扩散？

图像扩散模型处理的是连续像素，例如：

$x \in R^{H \times W \times 3}$

文本则是离散 Token：

```
我 → 123
喜欢 → 456
AI → 789
```

Token ID 是整数，不可以像普通图片像素一样随意加高斯噪声：

123+0.37=123.37

这个结果没有直接对应的 Token。

因此文本扩散有两条主要路线：

离散扩散（Discrete Diffusion）

在 Token / Mask 状态空间中添加和去除噪声。

连续扩散（Continuous Diffusion）

在连续 embedding 或 latent 表示空间进行扩散。

我们先从离散扩散讲起，因为它能直接和 Token、Cross Entropy、Transformer 联系起来。

三、离散扩散语言模型的数学原理

这一部分是核心。我们从一个最简单的 Masked Diffusion Model 开始。

3.1 定义 Token 空间

设词表大小为 VVV，文本长度为 LLL。

一段文本：

$$x_0 = (x_0^1, x_0^2, \ldots, x_0^L)$$

其中每个 Token：

$$x_0^i \in \{1, 2, \ldots, V\}$$

例如：

```
原始文本：我 喜欢 学习 AI
Token ID： [12, 45, 89, 102]
```

我们额外添加一个特殊 Token：

```
[MASK] = 0
```

所以扩散状态空间是：

$$\mathcal{X} = \{0, 1, 2, \ldots, V\}$$

这里的 0 代表被掩码的状态，不是普通词表中的 Token。

3.2 正向扩散过程（Forward Process）

我们定义一个噪声调度函数：

$$\alpha_t \in [0, 1]$$

它表示在时间 t 时，一个 Token 保持原始状态的概率。

通常：

$$\alpha_0 = 1, \qquad \alpha_1 = 0$$

也就是说：

t=0：没有噪声，全部 Token 保留。

t=0.5：大约一半 Token 被掩码。

t=1：全部 Token 被掩码。

对于每个 Token：

$$q(z_t^i|x_0^i) = \begin{cases} \alpha_t, & z_t^i = x_0^i \\ 1-\alpha_t, & z_t^i = \texttt{[MASK]} \end{cases}$$

整个序列的正向分布：

$$q(z_t|x_0) = \prod_{i=1}^{L} q(z_t^i|x_0^i)$$

这意味着每个 Token 独立地决定是否被 Mask。

3.3 一个具体例子

设：

x0=[我,喜欢,学习,AI]

取：

α t=0.5

随机 Mask 后可能得到：

```
x0 = [我, 喜欢, 学习, AI]
zt = [我, MASK, 学习, MASK]
```

再运行一次可能得到：

```
zt = [MASK, MASK, 学习, MASK]
```

不同随机结果都来自同一个正向扩散过程。

注意：这里的“噪声”不是高斯噪声，而是 Token 被替换为 MASK 的随机过程。

四、反向扩散：模型究竟预测什么？

训练一个 Transformer：

$$p_\theta(x_0|z_t, t)$$

它接收：

被掩码的序列 z_t 。

当前扩散时间 t。

输出每个位置的词表概率：

$$p_\theta(x_0^i|z_t,t)$$

例如：

```
输入：
我 [MASK] 学习 [MASK]

Transformer 输出：
位置 2：
喜欢 0.72
正在 0.12
想要 0.05
...

位置 4：
AI 0.81
机器学习 0.08
Python 0.04
...
```

这里有个非常重要的区别：

它可以同时预测所有位置，而不只是预测最后一个 Token。

这就是扩散式语言模型的主要生成机制之一。

五、训练目标：从 Cross Entropy 到 ELBO

这里我们深入到真正的论文数学。

5.1 最简单的训练目标：Masked Cross Entropy

给定干净文本 x_0 ，采样一个时间 t，得到被掩码文本 z_t。

模型输出：

$$p_\theta(x_0^i|z_t,t)$$

只对被 Mask 的位置计算交叉熵：

$$\mathcal{L}_{\text{mask}} = -\sum_{i=1}^{L} \mathbf{1}[z_t^i = \texttt{[MASK]}] \log p_\theta(x_0^i | z_t, t)$$

如果一个 Token 没有被 Mask，就不需要恢复它。

例子

真实文本：
我 喜欢 学习 AI

输入：
我 MASK 学习 MASK

预测：
位置 2 → 喜欢
位置 4 → AI

损失：

$$\mathcal{L} = -\log p_\theta(\text{喜欢}) - \log p_\theta(\text{AI})$$

如果模型预测概率：

p(喜欢)=0.8
p(AI)=0.9

则：

L=-log0.8-log0.9

约等于：

0.3285

概率越高，Loss 越低。

这和 BERT 的 Masked Language Modeling 很像，但扩散语言模型会在不同 Mask 比例下训练，并设计相应的生成过程。相关研究将这种目标进一步推导为扩散模型的似然训练目标。

5.2 为什么不能简单地只做一次 Mask？

如果训练时永远只 Mask 15%：

我 [MASK] 学习 AI

模型只习惯于恢复少量缺失 Token。

但推理时你可能从：

```
[MASK] [MASK] [MASK] [MASK]
```

开始。

这时模型遇到了训练中不常见的状态。

所以扩散语言模型通常让模型见过不同噪声程度：

```
Mask 0%   → 原始文本
Mask 25%  → 少量缺失
Mask 50%  → 一半缺失
Mask 75%  → 大部分缺失
Mask 100% → 全部缺失
```

训练目标需要对这些不同的噪声水平进行平均。

5.3 从概率模型理解 ELBO

扩散模型的完整概率建模方式是：

$$q(z_{1:T}|x_0) = \prod_{t=1}^{T} q(z_t|z_{t-1})$$

反向模型：

$$p_\theta(x_0, z_{1:T}) = p(z_T) \prod_{t=1}^{T} p_\theta(z_{t-1}|z_t)$$

这里：

q：人为定义的正向加噪过程。

pθ ：神经网络学习的反向去噪过程。

z0=x0 ：原始文本。

z_T ：最终纯噪声状态。

我们希望最大化：

$$\log p_\theta(x_0)$$

但是中间的隐变量 z1,…,zTz_1,\ldots,z_Tz1 ,…,zT 使得直接计算不容易，于是引入 ELBO（Evidence Lower Bound）。

$$\log p_\theta(x_0) \geq \mathbb{E}_q \left[\log \frac{p_\theta(x_0, z_{1:T})}{q(z_{1:T}|x_0)} \right]$$

这就是变分下界。

对于 Masked Diffusion Language Model，经过适当推导，可以得到非常接近传统 Masked LM 的损失形式。Sahoo 等人的 MDLM 工作提出了简化的训练目标，并展示了其与 Masked Language Modeling 的联系。

连续时间简化形式

在一个简单的 Mask 调度下，可以写成：

$$\mathcal{L}_{\text{MDLM}} = \mathbb{E}_{t\sim U(0,1)} \left[\frac{1}{t} \mathbb{E}_{z_t \sim q(z_t|x_0)} \left[-\log p_\theta(x_0|z_t) \right] \right]$$

这里的 ttt 可以理解为掩码比例对应的时间参数。不同论文可能采用不同的时间方向和调度定义，因此系数形式也可能不同。上式对应 Kuleshov Group 对简化 Masked Diffusion 目标的介绍。

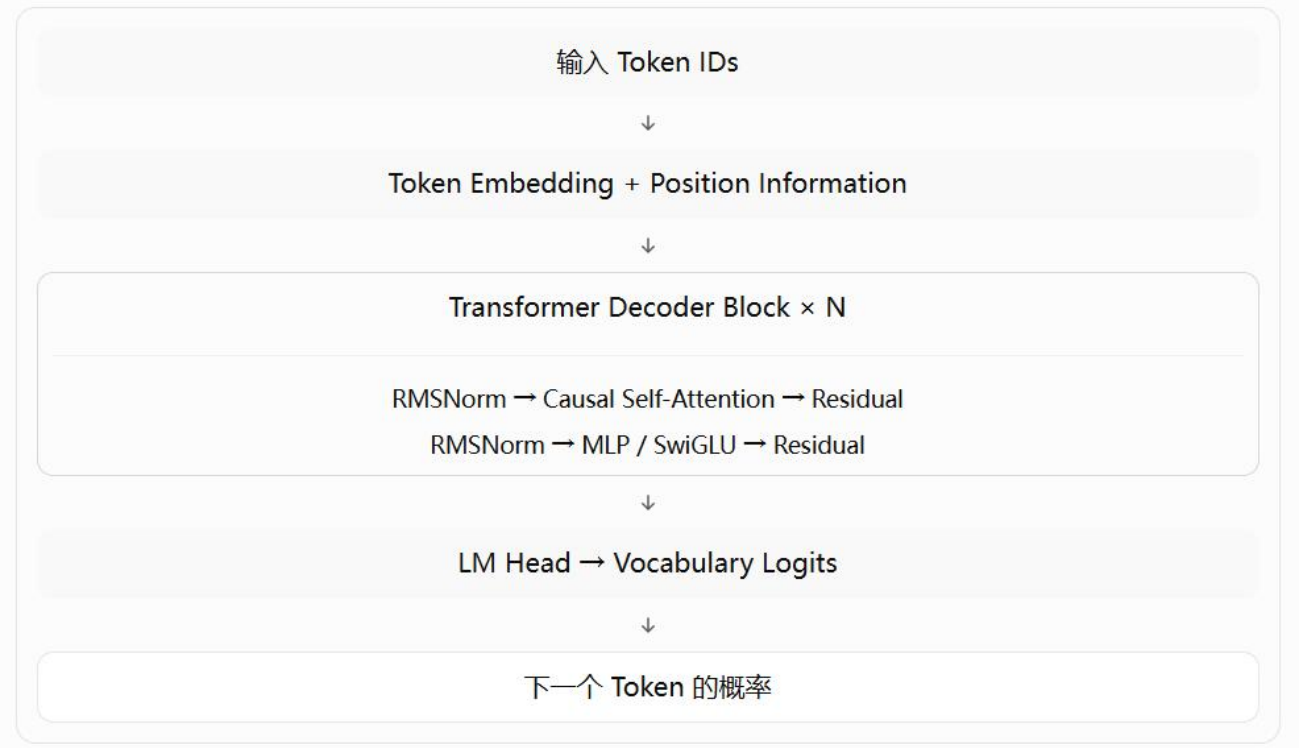
直观理解：训练不是只学一个固定 Mask 比例，而是学习一系列从高度破坏到轻度破坏的恢复任务。

六、Transformer 架构：扩散 LLM 到底长什么样？

这里是你之前学习 Qwen 结构最有联系的部分。

6.1 传统 Qwen Transformer

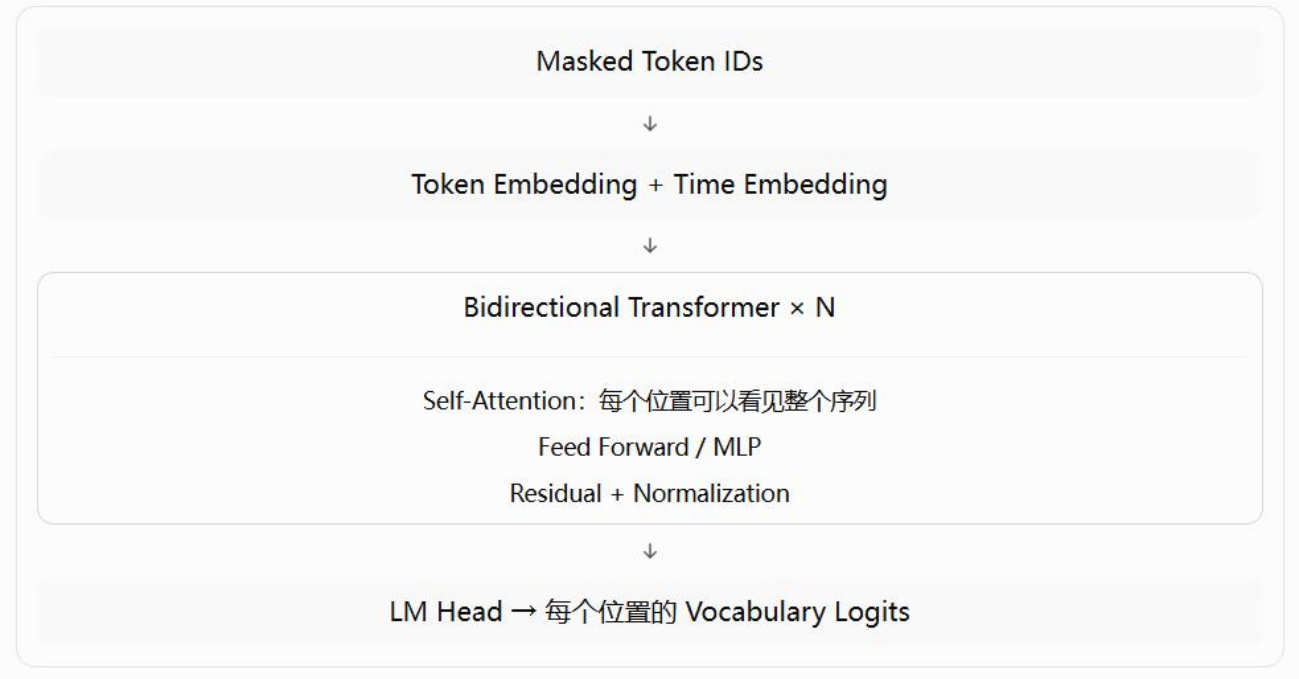
典型的 Decoder-only 自回归 LLM：



Qwen 系列具体的层数、隐藏维度、注意力头数等取决于版本和模型大小。这里的结构是传统 Decoder-only LLM 的概念示意。

6.2 扩散式语言模型的 Transformer

对于 Masked Diffusion LM，一种常见做法是使用双向注意力的 Transformer：



注意：这里的架构是一个典型的实现思路，不是所有扩散 LLM 的唯一架构。

为什么用双向 Attention？

传统 Qwen：

```
Token 1 → Token 2 → Token 3 → Token 4
```

```
Token 2 不能看 Token 3、Token 4
```

扩散 LM：

```
Token 1 ↔ Token 2 ↔ Token 3 ↔ Token 4
```

```
每个 Token 都可以看整个序列
```

因为在去噪过程中，模型需要利用上下文两侧的信息恢复缺失 Token。

例如：

```
我今天去 [MASK] 买了一杯咖啡
```

模型可以同时利用：

左边：“我今天去”

右边：“买了一杯咖啡”

来推断 `[MASK]` 可能是“星巴克”或者“商店”。

这也是为什么扩散 LM 的 Transformer 形态经常更接近 BERT/Encoder，而不是严格因果的 GPT Decoder。MDLM 论文明确讨论了使用 encoder-only 模型实现 Masked Diffusion Language Modeling。

七、Self-Attention 到底要不要改？

这是一个很容易混淆的点。

扩散模型并不要求发明一种新的 Attention 算法。

它主要改变的是：

输入是什么。

Attention Mask 是什么。

输出目标是什么。

推理时如何反复调用模型。

传统 Qwen 的 Attention Mask

对于序列：

```
A B C D
```

因果 Mask：

Query \ Key	A	B	C	D
A	✓	×	×	×
B	✓	✓	×	×
C	✓	✓	✓	×
D	✓	✓	✓	✓

扩散 LM 的双向 Attention Mask

Query \ Key	A	B	C	D
A	✓	✓	✓	✓
B	✓	✓	✓	✓
C	✓	✓	✓	✓
D	✓	✓	✓	✓

所以扩散 LM 的 Attention 通常可以使用普通双向 Self-Attention。

但在实际工程中，也有混合式模型，同时支持自回归和扩散模式。某些研究通过切换 Attention Mask，让同一个 Transformer 同时执行两种生成范式。

八、时间步 Embedding：扩散模型的重要新增输入

传统 Qwen 输入：

```
X=TokenEmbedding(x)
```

扩散 LM 还需要知道当前噪声程度。

例如：

```
输入：
我 [MASK] 学习 [MASK]

当前时间：
t = 0.5
```

同样的文本，在不同时间步，模型的任务不同：

t=0.1：少量 Mask，主要修复局部。

t=0.5：中等 Mask，需要理解上下文。

t=0.9：几乎全 Mask，需要生成整体结构。

因此模型输入：

H_0=E(x_t)+E_t(t)

其中：

E(x_t)：Token Embedding。

E_t(t)：时间步 Embedding。

时间 Embedding 可以通过 Sinusoidal Embedding、MLP 或其他方式实现。

九、从零写一个最小 Diffusion LM

下面代码用 PyTorch 实现一个教学版 Masked Diffusion Language Model。

它不是完整的工业级 MDLM，而是让你理解训练与推理的核心。

9.1 定义模型

```
import mathimport torchimport torch.nn as nnimport torch.nn.functional as F

class DiffusionLM(nn.Module):
    def __init__(
        self,
        vocab_size,
        hidden_dim=256,
        num_layers=4,
        num_heads=8,
        max_seq_len=128,
        mask_token_id=0,
    ):
        super().__init__()

        self.vocab_size = vocab_size
        self.hidden_dim = hidden_dim
        self.mask_token_id = mask_token_id
```

```
        # Token Embedding
        self.token_emb = nn.Embedding(
            vocab_size,
            hidden_dim
        )

        # Position Embedding
        self.pos_emb = nn.Embedding(
            max_seq_len,
            hidden_dim
        )

        # Time Embedding
        self.time_mlp = nn.Sequential(
            nn.Linear(1, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
        )

        # Bidirectional Transformer
        layer = nn.TransformerEncoderLayer(
            d_model=hidden_dim,
            nhead=num_heads,
            dim_feedforward=hidden_dim * 4,
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
        )

        # Vocabulary Prediction Head
        self.lm_head = nn.Linear(
            hidden_dim,
            vocab_size,
        )

    def forward(self, input_ids, t):
        """
        input_ids: [B, L]
        t:         [B]
        """

        B, L = input_ids.shape
```

```
        positions = torch.arange(
            L,
            device=input_ids.device
        )

        # Token Embedding
        h = self.token_emb(input_ids)

        # Position Embedding
        h = h + self.pos_emb(positions)[None, :, :]

        # Time Embedding
        t_emb = self.time_mlp(
            t[:, None]
        )

        h = h + t_emb[:, None, :]

        # Bidirectional Transformer
        h = self.transformer(h)

        # Logits: [B, L, vocab_size]
        logits = self.lm_head(h)

        return logits
```

代码重点

这个模型和传统 Qwen 的 Transformer 结构有很多相似之处：

Token Embedding。

Position Embedding。

Multi-Head Self-Attention。

Feed Forward。

LayerNorm / RMSNorm 类似的归一化思想。

LM Head。

真正重要的变化：

`t_emb = self.time_mlp(t[:, None])h = h + t_emb[:, None, :]`

以及：

```
h = self.transformer(h)
```

这里使用的是双向 Encoder 风格的 Transformer，而不是默认因果 Decoder。

9.2 实现正向 Mask 扩散

我们给每个 Token 随机决定是否变成 [MASK]。

```
def mask_diffusion(
    input_ids,
    t,
    mask_token_id=0,
):
    """
    input_ids: [B, L]
    t:         [B]
    """

    B, L = input_ids.shape

    # 每个位置被 mask 的概率
    mask_prob = t[:, None].expand(B, L)

    # 随机采样
    random_values = torch.rand_like(
        input_ids.float()
    )

    mask = random_values < mask_prob

    noisy_ids = input_ids.clone()

    noisy_ids[mask] = mask_token_id

    return noisy_ids, mask
```

例如：

```
input_ids = torch.tensor([
    [12, 45, 89, 102]
])
t = torch.tensor([
    0.5
])
noisy_ids, mask = mask_diffusion(
    input_ids,
    t,
```

```
    mask_token_id=0,
)
print(noisy_ids)
```

可能输出：

```
tensor([[12,  0, 89,  0]])
```

9.3 计算训练 Loss

```
def diffusion_loss(
    model,
    input_ids,
    mask_token_id=0,
):
    B, L = input_ids.shape

    # 随机采样时间步
    t = torch.rand(
        B,
        device=input_ids.device
    )

    # 正向扩散
    noisy_ids, mask = mask_diffusion(
        input_ids,
        t,
        mask_token_id,
    )

    # Transformer 预测所有位置
    logits = model(
        noisy_ids,
        t,
    )

    # 只对被 mask 的位置计算 CE
    loss_per_token = F.cross_entropy(
        logits.reshape(-1, model.vocab_size),
        input_ids.reshape(-1),
        reduction="none",
    )

    loss_per_token = loss_per_token.view(B, L)

    mask_float = mask.float()

    loss = (
```

```
        loss_per_token * mask_float
    ).sum() / mask_float.sum().clamp_min(1.0)

    return loss
```

这段代码的逻辑：

```
原始文本
    ↓
随机选择 t
    ↓
随机 Mask
    ↓
Transformer
    ↓
预测所有 Token
    ↓
只计算 Mask 位置的 Cross Entropy
```

训练循环

```
model = DiffusionLM(
    vocab_size=10000,
    hidden_dim=256,
    num_layers=4,
    num_heads=8,
).cuda()
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-4,
)
for step in range(10000):

    input_ids = get_batch().cuda()

    loss = diffusion_loss(
        model,
        input_ids,
    )

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    if step % 100 == 0:
        print(
            f"step={step}, "
```

```
            f"loss={loss.item():.4f}"
        )
```

注意：这个教学版训练目标和正式 MDLM 的严格离散扩散似然目标不完全相同。正式实现通常需要正确处理时间调度、反向转移概率和损失权重；上面代码主要展示 Masked Diffusion 的核心机制。

十、推理过程：如何真正生成文本？

训练模型之后，不能像普通 Qwen 一样直接：

```
model.generate(...)
```

因为我们没有标准的自回归 next-token 生成逻辑。

需要设计一个迭代去噪采样器。

10.1 最简单的 Masked Iterative Sampling

假设我们要生成 8 个 Token。

初始化：

```
[MASK] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK] [MASK]
```

然后多轮：

```
Step 1:
我 [MASK] 喜欢 [MASK] AI [MASK] 学习 [MASK]

Step 2:
我 喜欢 喜欢 学习 AI 模型 深度 学习

Step 3:
我 喜欢 学习 深度 学习 模型
```

实际算法通常还会考虑：

哪些 Token 置信度高。

哪些 Token 需要保留 Mask。

当前噪声水平。

反向扩散概率。

采样随机性。

10.2 教学版采样代码

下面是一个简单的“逐步填充”示例：

```
@torch.no_grad()def sample(
    model,
    batch_size,
    seq_len,
    mask_token_id=0,
    num_steps=20,
):
    device = next(model.parameters()).device

    # 初始全部 MASK
    x = torch.full(
        (batch_size, seq_len),
        mask_token_id,
        dtype=torch.long,
        device=device,
    )

    for step in range(num_steps):

        # 时间从高噪声到低噪声
        t = torch.full(
            (batch_size,),
            1.0 - step / num_steps,
            device=device,
        )

        logits = model(x, t)

        probs = torch.softmax(
            logits,
            dim=-1,
        )

        # 每个位置取最高概率 Token
        predicted = probs.argmax(dim=-1)

        # 置信度
        confidence = probs.max(dim=-1).values

        # 当前目标：保留一定数量 MASK
        keep_ratio = (
            1.0 - (step + 1) / num_steps
        )
```

```
        num_keep = int(seq_len * keep_ratio)

        if num_keep > 0:
            # 置信度最低的位置继续 MASK
            _, indices = torch.topk(
                confidence,
                k=num_keep,
                dim=-1,
                largest=False,
            )

            x = predicted.clone()

            x.scatter_(
                1,
                indices,
                mask_token_id,
            )

        else:
            x = predicted

    return x
```

这个采样器是为了展示思想，并不等于论文中的正式反向扩散采样器。特别是，严格的离散扩散模型需要按照反向转移分布采样，而不是简单地把低置信度 Token 保留为 MASK。

为什么需要多轮？

因为模型需要逐步从粗糙状态恢复到完整状态：

$$z_T \to z_{T-1} \to \cdots \to z_0$$

每一轮模型都利用当前上下文重新预测。

十一、Qwen 自回归和 Diffusion LM：深入对比

11.1 数学目标不同

Qwen / GPT

$$\mathcal{L}_{AR} = -\sum_{i=1}^{L} \log p_\theta(x_i | x_{<i})$$

例如：

输入：我 喜欢
预测：学习

每个位置只能依赖前面的 Token。

Diffusion LM

$$\mathcal{L}_{Diff} = \mathbb{E}_{t,z_t}\left[-\sum_{i \in M_t} w(t) \log p_\theta(x_i | z_t, t)\right]$$

其中：

M_t ：当前被 Mask 的位置。

w(t)：扩散时间对应的损失权重。

z_t ：被破坏的文本。

每个位置可以依赖整个带噪序列。

11.2 训练阶段对比

方面	Qwen 自回归	Masked Diffusion LM
输入	完整 Token 序列	带 MASK 的序列
Attention	Causal	通常 Bidirectional
预测	下一个 Token	被破坏 Token / 所有位置
时间 Embedding	通常没有扩散时间	需要噪声时间信息
Loss	Next Token CE	Masked CE / 扩散目标
一次训练样本	可计算各位置的 next-token loss	采样噪声状态，恢复 Token

11.3 推理阶段对比

方面	Qwen	Diffusion LM
初始状态	Prompt	Prompt + Masked positions

生成方式	逐 Token	多轮迭代恢复
KV Cache	很成熟	需要专门设计
并行性	受因果依赖限制	可以并行更新部分位置
采样步数	生成多少 Token 通常需要多少次模型步	需要若干轮去噪
流式输出	天然方便	需要特殊设计

十二、扩散 LM 的优势和问题

优势

1. 有机会减少串行依赖

传统自回归：

x1→x2→x3→…

扩散式：

整段文本→多位置并行修复

这为减少生成延迟提供了可能。

2. 更自然的全局修订

例如：

我想去北京旅游三天。

如果要把“三天”改成“五天”，扩散式生成可以把它视为整体文本的一部分重新修复，而不是只能从头到尾生成。

3. 与 Masked LM 有联系

BERT 的训练方式可以作为理解扩散 LM 的起点。相关工作已经证明，简单的 Masked Diffusion 目标可以用于训练生成式模型，而不仅仅是理解型模型。

问题

1. 推理不一定更快

如果每次去噪都需要运行完整 Transformer：

20 次去噪 × 每次完整模型计算

可能比传统 Qwen 生成 20 个 Token 还慢。

所以真正的速度优势取决于并行程度、去噪步数、模型规模和硬件。

2. 文本的离散空间比较难处理

图片加高斯噪声很自然，Token 则不是连续值。

因此 Masked Diffusion 是一种解决方案，不是唯一方案。

3. 生成质量和采样算法强相关

扩散 LM 的最终质量不仅取决于 Transformer，还取决于：

Noise schedule。

Reverse process。

Sampling algorithm。

Loss weighting。

Tokenizer。

模型规模。

4. 长文本和复杂推理仍然是挑战

不能简单认为“可以并行生成”就意味着一定比 Qwen 更擅长复杂推理。需要通过实验比较。

十三、如果你要用 Claude Code 实现一个 Diffusion LM 项目

你现在已经在学习 Transformer、Qwen、RAG 和模型训练，我建议按下面的路线实现，而不是一开始就尝试训练超大模型。

阶段 1：最小可运行模型

目标：验证扩散 LM 的核心机制。

PyTorch。

词表 1,000～10,000。

Transformer Encoder。

Masked CE。

固定长度序列。

简单 Mask 采样。

训练 Loss 曲线。

阶段 2：严格离散扩散实现

目标：从教学代码走向论文实现。

Forward masking process。

Reverse transition。

时间调度。

ELBO / MDLM loss。

正确的采样器。

Perplexity / NLL 评估。

阶段 3：和 Qwen 做对比

目标：研究价值更高。

同一份数据、相似模型规模，比较：

指标	Qwen 自回归	Diffusion LM
Validation Loss	测量	测量
Perplexity	测量	测量
生成速度	测量	测量
生成质量	测量	测量
长文本质量	测量	测量
显存	测量	测量

如果你想研究这个方向，MDLM 是一个很好的起点。论文和代码资源可以从 NeurIPS 2024 的 MDLM 论文页面 和 OpenReview 论文全文 开始。

十四、最终理解：它不是“换一种 Attention”，而是换一种生成范式

把这四个概念记住就够了：

1. Forward Diffusion

把原始文本随机 Mask，得到带噪序列。

2. Denoising Transformer

根据带噪文本和时间步，预测原始 Token。

3. Diffusion Training Objective

在不同噪声水平下训练恢复能力，常与 Masked CE / ELBO 有关。

4. Iterative Sampling

从高噪声状态开始，重复调用模型逐步恢复完整文本。

一句话总结：

> Qwen 的核心是学习 p(xi｜x<i)p(x_i|x_{<i})p(xi ｜x<i)，即根据前文预测下一个 Token；Diffusion LM 的核心是学习 p(x0｜zt,t)p(x_0|z_t,t)p(x0 ｜zt ,t)，即根据带噪文本和噪声时间恢复原始 Token。Transformer 可以是相似的，但训练目标和生成过程完全不同。

推荐继续研究

如果要真正理解论文里的数学，下一步最值得做的是：

1. 详细推导离散扩散的 ELBO

从 $q(z_t|x_0)$ 和 $p_\theta(z_{t-1}|z_t)$ 推导到实际 Loss。

MDLM 论文全文 ↗

2. 从零实现一个严格 MDLM

完整 PyTorch 项目：训练、验证、采样、Loss 曲线。

MDLM 论文与代码入口

扩散模型的应用

DLM（Diffusion Language Model，扩散语言模型）目前更适合以下几类应用。它的核心优势不是简单地“逐字生成”，而是可以同时生成、反复修改多个位置的 token。

1. 文本补全与局部填空

例如：

```
我今天去了 _____，天气非常好。
```

DLM 可以直接预测中间缺失的内容，而不是必须从左到右生成整句话。

具体应用：

- 代码补全
- 文档补全
- 邮件/报告补句
- 长文本缺失片段恢复
- Cloze 测试
- 根据上下文填充表格字段

例如代码：

```
def calculate_total(items):
    # DLM 补全这里的代码
```

它可以根据函数前后文，同时推断中间缺失的多行代码。

2. 代码编辑与代码重构

这是 DLM 很有潜力的方向。

传统自回归模型通常是：

```
读取旧代码 → 从头生成修改后的代码
```

DLM 可以更接近：

保留大部分原代码 → 对需要修改的区域进行扩散/去噪 → 得到新代码

适合：

修改某个函数

修复 bug

代码重构

API 替换

变量名批量修改

增加异常处理

补充类型注解

根据测试失败信息修改代码

IDE 中的局部代码编辑

例如：

```
def login(user, password):
    return db.query(
        f"SELECT * FROM users WHERE name = '{user}'"
    )
```

DLM 可以只修改 SQL 查询部分，变成参数化查询，而不需要重新生成整个文件。

这与 IDE 中的“局部代码编辑”非常匹配。

3. 文本改写和编辑

DLM 天然适合“保留原文结构，只修改一部分”。

例如：

改变语气

原文：
这个方案不太好。

要求：
改成正式、委婉的商务表达。

修改风格

普通中文 → 学术中文

中文 → 技术文档

长句 → 简洁表达

口语 → 正式表达

文章润色

语法纠错

敏感词替换

指定段落重写

因为它可以对多个位置反复修改，所以在“编辑已有文本”上可能比纯从左到右生成更自然。

4. 机器翻译与多语言转换

DLM 可以把翻译看成一个整体优化问题：

源语言：
`The system failed because the database connection timed out.`

目标语言：
系统失败，因为数据库连接超时。

它可以同时考虑：

整体语义

词序

上下文

长距离依赖

术语一致性

尤其适合：

长句翻译

技术文档翻译

多语言代码注释

双语文档生成

术语约束翻译

不过，目前自回归翻译模型的工程生态仍然更加成熟。

5. 受约束文本生成

如果输出必须满足某些结构，DLM 可能比较有优势。

例如生成：

```
{
  "name": "...",
  "age": 30,
  "skills": ["Java", "Python"]
}
```

约束包括：

必须是合法 JSON

必须包含指定字段

字段顺序或格式固定

某些 token 必须出现

某些词不能出现

输出长度受限

多个字段之间必须保持一致

DLM 可以把整个输出视为待优化的 token 序列，再反复修正不符合约束的部分。

但要注意：DLM 不会自动保证 100% 合法 JSON，仍然需要 grammar constraint、结构化解码或后处理。

6. 文本生成与规划

DLM 也可以用于：

生成故事大纲

生成文章结构

生成任务计划

生成多个候选答案

生成产品需求文档

生成实验方案

生成对话回复

例如要求：

```
请生成一个包含：
1．问题背景
2．技术方案
3．实施步骤
4．风险分析
5．验收标准

的技术方案。
```

DLM 可以先形成整体草稿，再多轮修改不同段落，使内容更加一致。

不过，在复杂推理、工具调用、长链路 Agent 任务上，当前主流自回归模型通常更成熟。

7. 交互式写作和实时编辑

DLM 特别适合类似编辑器的场景：

Word 智能改写

Notion AI 文档编辑

邮件润色

在线 IDE

AI 写作助手

Markdown 文档优化

论文段落重写

简历优化

用户选中一段文字：

这段内容太啰嗦了，请简化。

模型只需要修改选中区域，其他内容保持不变。

这种模式可以称为：

Infilling / Inpainting / Text Editing

其中 inpainting 原本常见于图像扩散模型，意思是“修补指定区域”；在文本中就是“修补或重写指定 token 区域”。

8. 可能用于更高效的并行生成

自回归模型是：

```
token 1 → token 2 → token 3 → token 4
```

DLM 可以尝试：

```
同时预测多个 token
→ 检查不确定位置
→ 重新去噪
→ 得到完整文本
```

例如：

```
第 1 轮：我 ___ 去了 ___，天气 ___ 好
第 2 轮：我今天去了北京，天气非常好
第 3 轮：修正部分不确定 token
```

理论上可以减少严格的逐 token 串行依赖。

但这里要特别注意：

DLM 不一定天然比自回归模型快。

因为它可能需要：

多轮去噪

多次 Transformer 前向计算

反复检查 token

额外的采样或置信度判断

只有在并行生成带来的收益超过多轮迭代成本时，才可能更快。

与应用对应的模型类型

应用	DLM 适配程度	原因
代码补全	很高	可以根据前后代码补中间内容
代码重构	很高	只修改局部代码
Bug 修复	很高	可以反复修改错误区域
文本润色	很高	保留原文并编辑指定部分
机器翻译	较高	可以整体考虑词序和语义
结构化 JSON 生成	较高	可反复修正结构
长篇自由聊天	中等	自回归生态更成熟
多轮 Agent	目前通常较低	工具调用和状态管理更适合自回归
数学推理	取决于模型	需要专门训练，不是扩散结构自动带来优势
实时流式聊天	自回归更成熟	DLM 通常不是天然逐 token 输出

具体展品

DLM 还非常少，ChatGPT、Claude Code、Cursor 默认使用的主力模型都不是 DLM。真正商业化、比较有代表性的 DLM 产品，目前最值得关注的是 Inception Labs 的 Mercury 系列。

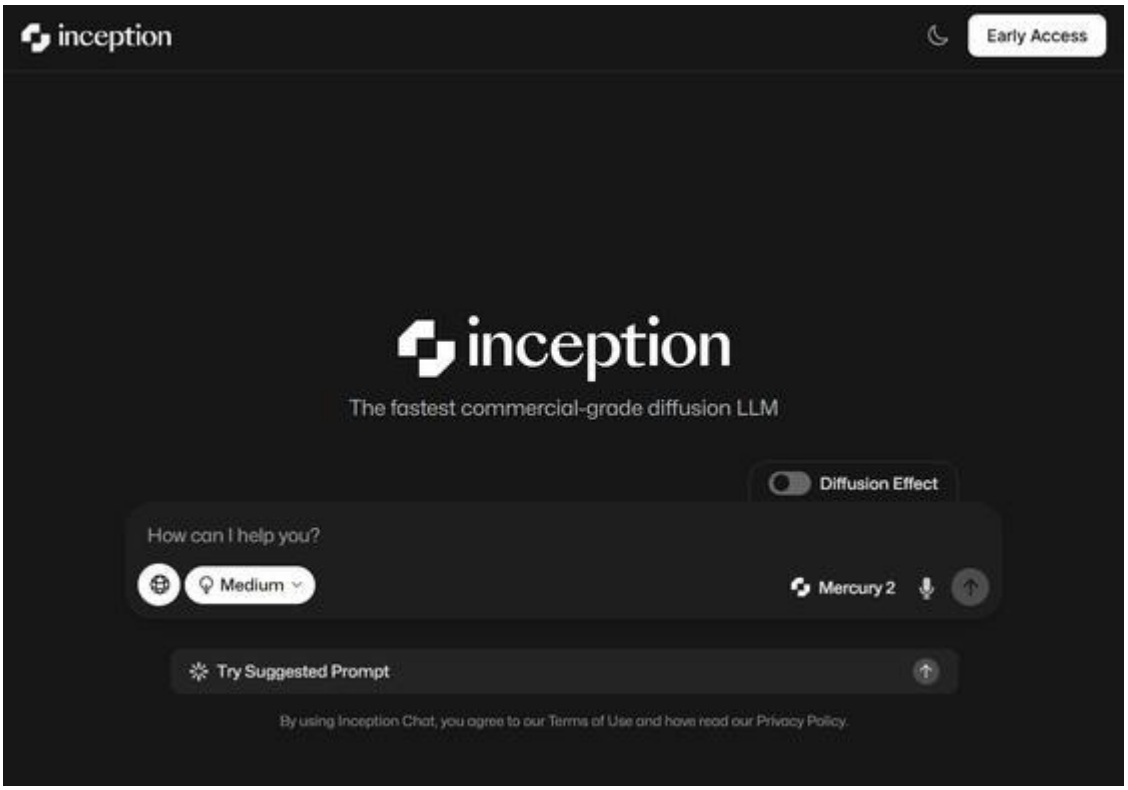
具体产品对照

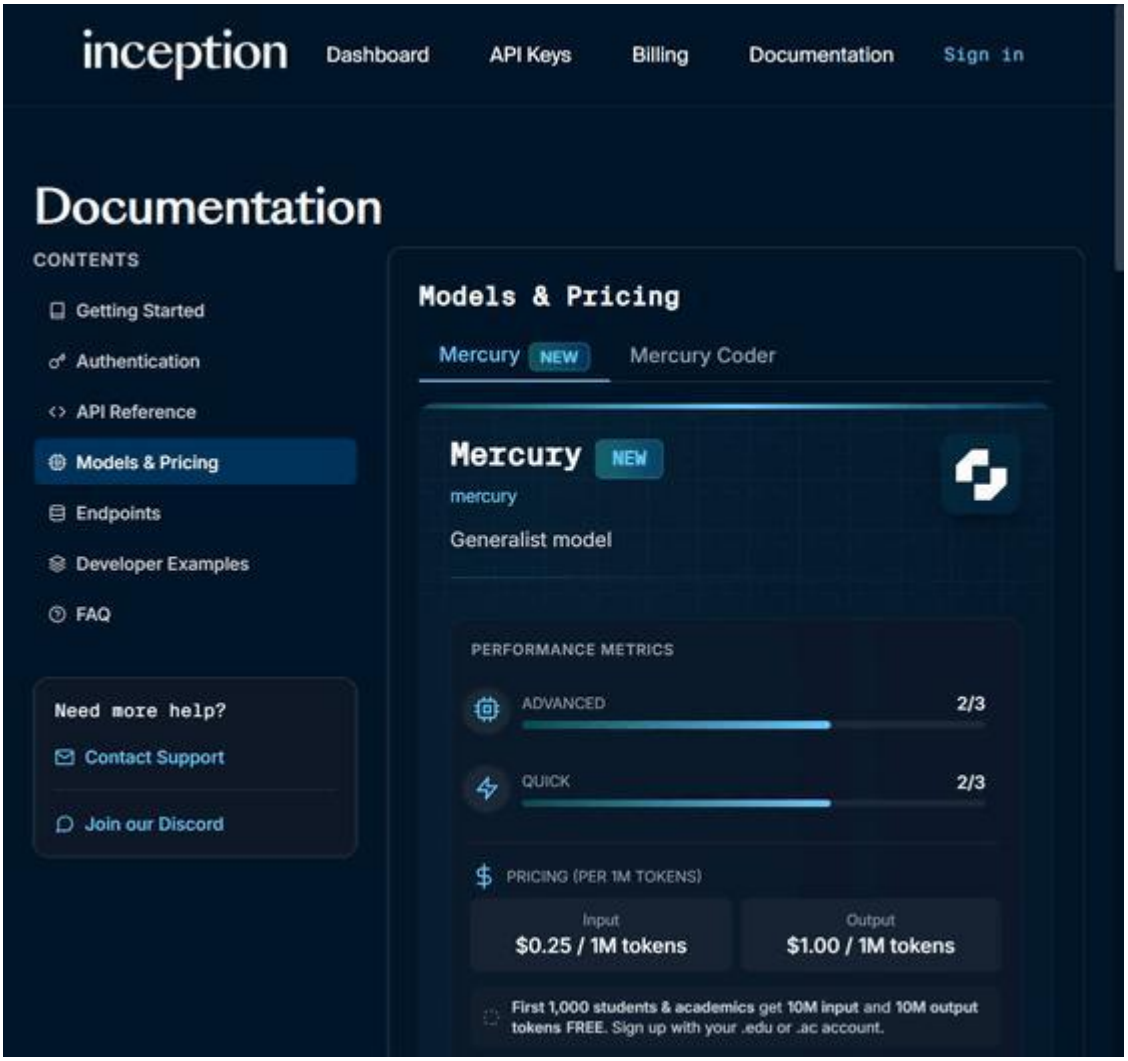
产品/模型	DLM?	主要模型路线
ChatGPT	✗	自回归为主
Claude / Claude Code	✗	自回归
Cursor	✗	本身是 AI 编程产品，背后可调用多种模型，主流仍是自回归
GitHub Copilot	✗	主流模型仍是自回归
Gemini	✗	主流是自回归/Transformer 路线
Qwen	✗	自回归
DeepSeek	✗	自回归
Llama	✗	自回归
Mercury	✓ DLM	Diffusion LLM
Mercury Coder	✓ DLM	Diffusion LLM，代码专用
Mercury 2 / 2.5	✓ DLM	Diffusion LLM

其中 **Mercury Coder** 是目前特别值得你关注的，因为它直接瞄准了 **Cursor / Copilot / Claude Code 所处的代码生成市场**。Inception 官方称它是 diffusion-based coding model，并提供 FIM、Next-Edit、Apply-Edit 等代码场景能力。

---

1. Mercury：真正的 DLM 产品





Inception Labs 推出的 **Mercury** 是目前最典型的商业化 DLM。

它不是：

> “一个 ChatGPT 外壳 + DLM”

而是**底层模型本身就是 Diffusion Language Model**。

传统模型：

```
token 1
   ↓
token 2
   ↓
token 3
   ↓
token 4
   ↓
...
```

Mercury：

```
噪声文本
    ↓
第一次去噪
    ↓
第二次去噪
    ↓
第三次去噪
    ↓
完整答案
```

而且一次可以处理多个 token，因此官方重点宣传的是**速度**。Mercury Coder 在 H100 上官方公布过超过 1000 tokens/s 的吞吐。

---

2. Mercury Coder：最值得你关注

如果你主要关心 **Claude Code / Cursor / AI 编程**，那么你应该重点看：

```
Mercury Coder
```

它就是专门做代码的 DLM。

它支持：

```
代码生成
代码补全
Fill-in-the-Middle
Next-Edit
Apply-Edit
代码修改
代码重构
Bug 修复
```

尤其是：

Fill-in-the-Middle

例如：

```
def login(user):
    # 这里是未知区域

    return result
```

模型同时看到前面的：

```
def login(user):
```

和后面的：

```
return result
```

然后生成中间代码。

这就是 DLM 非常适合的场景。

Inception 后续又加入了 **Next-Edit / Apply-Edit**，也就是直接预测“接下来应该修改哪里、怎么修改”，这已经非常接近现代 AI Coding Agent 的核心工作流。

---

3. 那 Cursor 是不是 DLM?

不是。

这个地方特别容易搞混。

Cursor 本身更准确地说是：

$$\text{AI IDE / AI Coding Agent}$$

而不是一种语言模型架构。

它可以调用不同模型。

所以：

```
Cursor
   │
   ├── Claude
   ├── GPT
   ├── Gemini
   ├── 自定义模型
   └── 其他模型
```

因此：

$$\text{Cursor} \neq \text{DLM}$$

但理论上 Cursor 完全可以接入 Mercury 这种 DLM。

实际上 Inception 官方也强调 Mercury Coder 可以通过 API 接入开发工具和 IDE。

---

4. Claude Code 是不是 DLM?

不是。

Claude Code 是：

```
Coding Agent / CLI
```

它背后的 Claude 模型是自回归 Transformer 路线。

所以不要把：

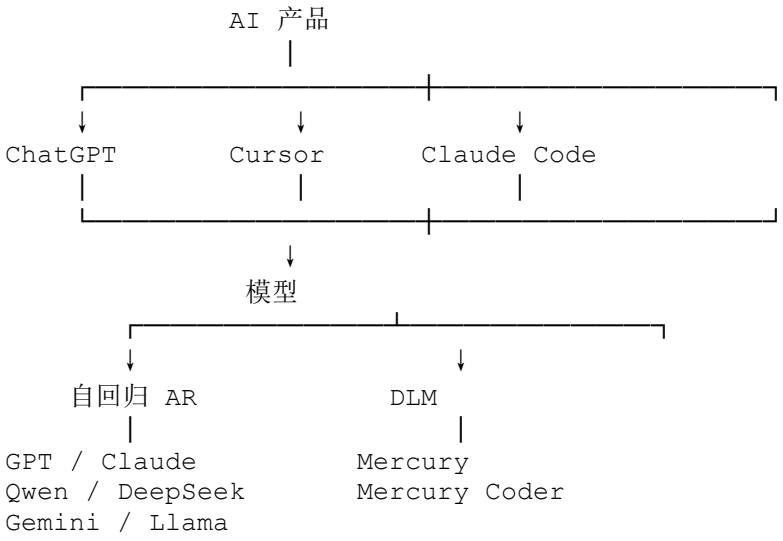
```
Claude Code
Cursor
Codex
GitHub Copilot
```

和：

```
GPT
Claude
Qwen
DeepSeek
Mercury
```

放在同一个层级比较。

更准确的结构是：



---

5. 目前真正值得记住的 DLM 名字

如果你是在学习这个领域，我建议你现在只记住几个：

第一梯队

Inception Labs Mercury

商业化 DLM 的代表。

Mercury Coder

DLM + Coding，和你的 Claude Code / Cursor 方向最相关。

Mercury 2 / Mercury 2.5

更通用、更强的新一代 Mercury。

目前 Inception 已经把 Mercury 用于 coding、search、voice、agent 等低延迟场景。

---

6. 一个非常重要的变化

你前面问：

“ChatGPT 是自回归还是 DLM？”

答案是：

ChatGPT → 自回归。

但现在这个领域正在出现一个非常有意思的竞争：

传统路线

```
GPT / Claude / Gemini / Qwen / DeepSeek
                ↓
        Autoregressive
                ↓
        一个 token 一个 token
```

VS

新路线

```
Mercury
   ↓
Diffusion LLM
   ↓
多个 token 并行
```

↓
反复 refinement
↓
快速生成

而且 Mercury 已经不是纯论文实验了，已经有 API、代码模型、通用聊天模型以及企业部署。

所以如果你现在正在系统学习 LLM，我建议你把 DLM 当成“下一条值得重点观察的 LLM 架构路线”，而不是把它理解成 ChatGPT 已经在使用的技术。

如果你愿意，我下一步可以直接给你讲 **“Mercury DLM 到底是怎么做到 1000+ tokens/s 的：从训练目标 → Masking → Transformer → 并行去噪 → KV Cache → 推理过程，用代码把它拆开”**。

Mercury 系列产品还不能跟 chatgpt 或 claude 比吧？所以 dlm 现在还是很弱

对，**你的判断基本正确，但要加一个很重要的限定：DLM 目前不是“整体很弱”，而是“顶级通用智能仍明显落后于最强的自回归模型，但在速度/延迟这个维度非常有竞争力”。**

而且截至 **2026 年 9 月**，DLM 已经比我们刚才讨论时成熟了一大截。

直接看当前水平

以最新的 **Mercury 2.5** 为例，Inception 官方现在把它定位为目前最强的 diffusion LLM，并称其质量已经达到 **成本优化型 frontier 模型**的水平，同时保持约 **1,107 tokens/s**。

但这里的关键词是：

> cost-optimized frontier models

而不是：

> GPT / Claude 最顶级模型

所以如果你问：

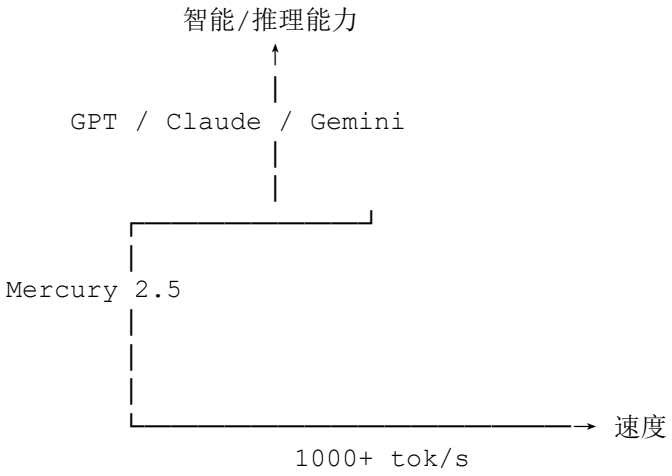
> “Mercury 2.5 能不能全面替代 GPT-5.x / Claude Sonnet/Opus 级别模型？”

我的判断仍然是：

不能。

为什么会出现这个现象？

可以把现在的 LLM 竞争粗略画成：



也就是说，DLM 当前最明显的优势不是：

> “我比 GPT 更聪明。”

而是：

> **“我可以用接近可用的智能，极快地生成大量 token。”**

这其实是 DLM 当前最大的价值

例如 Mercury 2.5 官方公布：

1,107 tokens/s

这意味着：

```
100 tokens   ≈ 0.09 秒
500 tokens   ≈ 0.45 秒
1000 tokens  ≈ 0.90 秒
```

这和传统自回归模型完全不是一个体验。

而对于 Agent：

```
用户
 ↓
Agent
 ↓
```

```
LLM
 ↓
Tool
 ↓
LLM
 ↓
Tool
 ↓
LLM
 ↓
...
```

如果一次 Agent 要调用模型几十次，那么每一次减少几百毫秒甚至几秒，最终体验差距会非常大。

这也是为什么 Mercury 现在重点攻：

- Search Agent
- RAG
- Voice Agent
- Coding
- 实时交互
- 高频 Agent loop

而不是单纯跟 GPT/Claude 拼“谁的数学推理最强”。

更关键的是：DLM 的弱点不是“DLM 天生不聪明”

这个一定要区分。

DLM ≠ 低智力模型。

目前的问题更接近：

```
Transformer + 自回归
        ↓
已经发展了很多年
        ↓
Scaling / RL / RLHF / SFT / Reasoning
        ↓
GPT / Claude / Gemini
        ↓
非常成熟
```

而：

```
Transformer + Diffusion
        ↓
语言领域仍然非常新
        ↓
训练方法还在探索
        ↓
推理/采样/对齐/长程规划仍在优化
        ↓
Mercury
```

所以现在拿：

发展多年的 GPT/Claude

去对比：

刚开始商业化的 DLM

其实有点不公平。

但有一个特别有意思的信号

现在 DLM 已经开始在某些任务上**不是弱，而是非常强**。

例如 Inception 最近公布的 Mercury 2 搜索 Agent 实验：

- FRAMES：Mercury 2 与其他模型基本处于同一水平
- DeepSearchQA：Mercury 2 为 0.34，而 GPT-5 Mini 为 0.44
- 在一些需要检索 + 综合的任务上，Mercury 2 可以达到接近 frontier 模型的效果

而速度优势依然非常明显。

另外在 PinchBench 的 Agent 测试里，Inception 报告 Mercury 2 达到 **78% success rate**，高于其对比中的 GPT-5 Mini 75%、Gemini 2.5 Flash 71%、DeepSeek Chat 72%、GPT-4o 71%。当然，这是厂商自己的测试，应该谨慎看待，不能直接等价成“Mercury 比 GPT-5 强”。

所以我会这样评价 DLM

能力	当前 DLM
普通聊天	已经可用
简单代码	很强
Code completion	**非常有潜力**

能力	当前 DLM
Next Edit	**非常适合**
RAG	很有竞争力
高频 Agent	**非常有优势**
实时语音	**非常有优势**
长文本生成	
复杂数学	
深度推理	/🟢
顶级 Coding Agent	🟡 目前还不是 Claude Code/Codex 的全面替代品
顶级通用智能	🟡 暂时落后
生成速度	**巨大优势**

所以你可以记住一句话

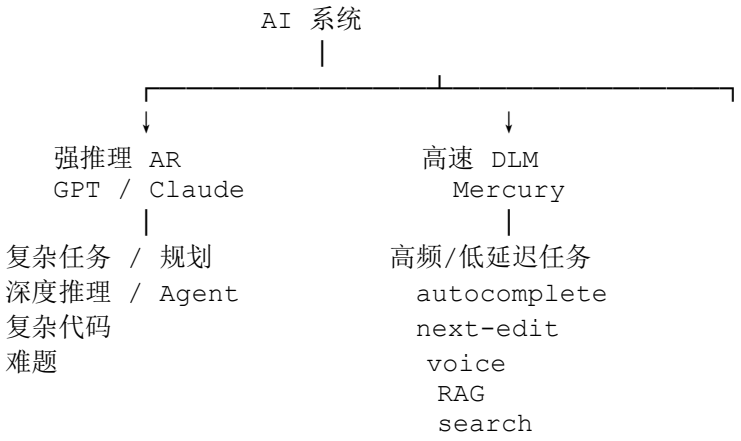
现在的 DLM 并不是“另一个 GPT”，而更像是：

用一种全新的生成方式，把“足够强的模型”做到极低延迟。

目前最合理的路线甚至可能不是：

```
DLM 取代所有 AR
```

而是：



这反而是我认为 DLM 目前最现实、也最有商业价值的方向。

而且你如果从 **Claude Code / Cursor / AI Coding Agent** 的角度看，DLM 其实特别值得关注：**它不一定首先替代 Claude Code 的“大脑”，但非常可能先替代其中那些“高频、低延迟的小动作”——autocomplete、next-edit、代码局部修改、快速工具调用。** Mercury Edit 2 目前已经直接针对这个场景，并且已经接入 Zed 等开发工具。

如果你想深入理解，我下一步最建议讲的是**“为什么 DLM 可以 1000 tok/s，而 GPT/Claude 只能几十到几百 tok/s？到底是哪里省掉了计算？”**这个问题一旦搞懂，DLM 和自回归的本质区别就真正明白了。